CREPE-YOLO: A YOLO-BASED APPROACH FOR AUTOMATIC MONOPHONIC MUSIC TRANSCRIPTION

Leonid Alexeyev, Disha Baglodi, & Spencer Kessler

1 INTRODUCTION

Deep learning models have made much progress with the task of automatic music transcription, in which a musical audio input is processed to produce a MIDI file corresponding to the notes of the audio. CREPE is a model that inputs musical audio and predicts a continuous f_0 contour, which is a spectrogram describing the audio. However, the transcription is incomplete: to get the discrete notes, CREPE requires a manual post-processing step to produce a MIDI output.

To perform this post-processing, a heuristic called CREPE Notes has been proposed by Bresin & Falkenberg (2023), in which the f_0 contour outputted by CREPE is processed through a deterministic algorithm to produce a MIDI file of the discrete notes. While this method does perform well compared to various other transcription methods, the use of determinism to solve a nondeterministic problem could be a hindrance to the algorithm's adaptability.

To improve upon this process, we propose a deep learning model that implements a post-processing step of CREPE's output. After processing a monophonic audio input through CREPE, our model will take as input the outputted spectrogram and predict the discrete notes as a MIDI file.

To provide an accurate comparison with CREPE Notes, we plan to use the same datasets used by Bresin & Falkenberg (2023), which are the ITM-Flute-99 (Köküer et al. (2019)) and Filosax (Foster & Dixon (2021)) datasets. All training and experiments will be conducted on the GT PACE ICE cluster. Our model extends a pre-trained frozen CREPE backbone, and adds a lightweight temporal decoding model, keeping computational requirements tolerable. We use the H100 and H200 GPUs for efficient batched training and hyperparameter tuning.

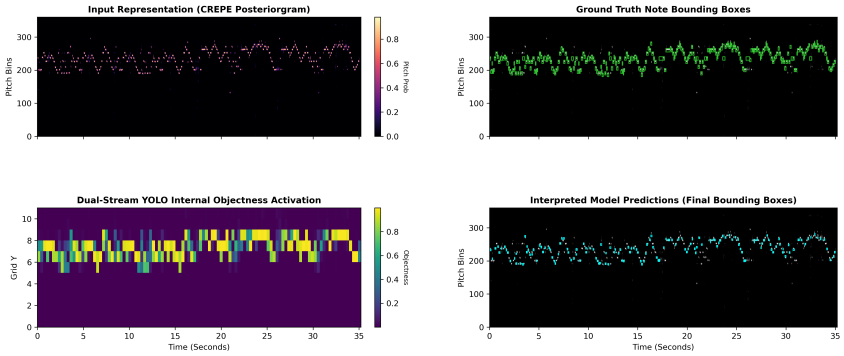

Figure 1: Top Left: CREPE output. Top Right: Ground truth bounding boxes over CREPE output. Bottom Left: Objectness Activation (confidence) from YOLO. Bottom Right: Predicted bounding boxes over CREPE output.

2 RELATED WORK

In order to construct a note recognition pipeline, frame-wise pitch must first be estimated from an audio sample. Prior to machine learning approaches, this was done via heuristic algorithms, among which probabilistic YIN (pYIN) achieved strong performance. Proposed by Mauch & Dixon (2014), pYIN extends the classic YIN pitch detection algorithm by outputting multiple pitch candidates per frame and applying a hidden Markov model to smooth these into a frequency track. The introduction

of probabilistic distributions and sequential decoding improves precision and recall over traditional methods, at a low additional complexity.

An alternative data-driven approach to the pitch detection problem was introduced with CREPE, where Kim et al. (2018) proposed a convolutional neural network (CNN) for monophonic pitch estimation from raw audio waveforms, without manual heuristic spectral features. This CNN predicts fundamental frequency using a high-resolution pitch bin representation, and was shown to perform better or equally than previous best-performing techniques (e.g. pYIN). However, CREPE produces continuous f_0 estimates rather than discrete musical note events or temporal structure.

One end-to-end model that performs polyphonic piano transcription is described in Kwon et al. (2020), in which the model takes in piano audio as input, converts it into a log-mel spectrogram, and then is processed through a series of modules to output a MIDI file. The modules include a CNN-based model to extract local features followed by an RNN-based autoregressive model to predict the note states given the previous states and the CNN output. These states are then decoded to produce a MIDI output. While this model's limitation to piano transcription will differ from our model, other elements of this model may still be applicable to our use case.

A more recent model called MT3 takes a log-mel spectrogram as input and transcribes it into a polyphonic MIDI output with capabilities to support multiple instruments Gardner et al. (2022). It uses a transformer architecture, in which the input is encoding using a series of self-attention blocks, which are also used for autoregressive decoding. After the model outputs an end-of-sequence token, the output tokens are able to be detokenized into MIDI data through a deterministic process. While surpassing SOTA baselines, CREPE Notes was able to surpass MT3. However, MT3 does perform more complex tasks than CREPE Notes, so we may still be able to take inspiration from MT3's approach by scaling it down for monophonic transcription.

Addressing segmentation from a computer vision perspective, MusicYOLO Wang et al. (2022) recently proposed treating note transcription as an object detection task rather than a frame-wise classification task. By converting audio into Constant-Q Transform (CQT) spectrograms, MusicYOLO utilizes a YOLOX Ge et al. (2021) backbone to construct bounding boxes representing note onset, offset, and pitch. This approach was shown to reduce fragmentary errors common in frame-based models and achieved state-of-the-art F1-scores on vocal datasets.

3 Problem Decomposition & Detailed Technical Approach

3.1 Neural Network Inputs and Outputs

The neural network for this project will be take as input raw audio waveforms, specifically monophonic instrument recording. The model will output a tensor of note bounding boxes defined by (time onset, time offset, pitch) and a confidence score.

This network architecture will be made up of multiple modules:

- **CREPE (Potentially Frozen)**:
  - *Input:* Raw audio waveform (16kHz).
  - *Output:* 360-bin posteriorgram (pitch probability map), 2048-dimension embedding, and 1-dimension confidence score.
- **Dual-Stream Encoder Branches**:
  - *Branch A (Visual Stream – 2D-CNN)*:
    - *Input:* 360-bin posteriorgram concatenated with a CoordiConv channel (normalized vertical frequency coordinates $0.0 - 1.0$).
    - *Operation:* 2D-Convolutional backbone for spatial feature extraction.
    - *Output:* 2D Visual Feature Map.
  - *Branch B (Semantic Stream – CRNN)*:
    - *Input:* Stacked 2050-dimension vector (2048 embedding + confidence + calculated pitch gradient).

    - *Operation:* 1D-Convolutional dimensionality reduction (bottleneck) followed by a Bi-Directional LSTM for temporal smoothing.
    - *Output:* Compressed Semantic Feature Vector (e.g., 64 channels).
- **Fusion and Tiling Module**:
  - *Input:* 2D Visual Feature Map and 1D Semantic Feature Vector.
  - *Operation:* Vertical broadcasting (tiling) of the Semantic Vector to match the height of the Visual Map, followed by concatenation.
  - *Output:* Unified Spatio-Semantic Feature Block.
- **YOLOX Detection Head**:
  - *Input:* Unified Spatio-Semantic Feature Block.
  - *Output:* Tensor of note bounding boxes confidence scores.

3.2 Loss Function

We will use the loss from the YOLOX structure, this is a multi-part loss function made up of the following two components:

- *1D IoU Loss:* Intersection-over-union error between the predicted bounding box and the ground truth note duration/pitch.
- *Objectness Loss:* Binary cross-entropy loss that penalizes the model for detecting notes in silence or missing notes entirely.

3.3 Quantitative Evaluation Metric:

To have comparability with CREPE Notes, Pyin, and MT3, we will evaluate the model using the standard *mir_eval* library. The metrics will be:

- *Precision & Recall:* Measure the model's ability to correctly detect note events while minimizing false positives.
- *F-Measure:* The harmonic mean metric used to benchmark overall transcription performance.
- *Average Overlap Ratio:* The segmentation accuracy (duration alignment) of the detected notes.

3.4 Evaluation Procedure

For each experiment, we split the dataset into 64% train, 16% validation, and 20% test sets. During hyperparameter tuning, we trained each configuration on the training split, evaluating at the end of each epoch on validation and test sets. We selected the best checkpoint using the validation F1-score, and used the test set as a held-out benchmark for final comparison.

In addition, we performed a training set size ablation by retraining the best hyperparameter configuration with 10%, 20%, ..., 100% of the available training data to see if adding more labeled data is likely to improve performance (to predict the results of adding the Filosax dataset). We also performed and evaluated the effects of data augmentation, including pitch-shifting and tempo modulating training examples to improve generalization under limited data.

For evaluation, predicted note boxes are converted into note events and compared against ground-truth annotations using mir_eval transcription metrics. Using this, we report precision, recall, F-measure, and average overlap ratio (AOR) in order to compare directly against CREPE Notes and other baselines.

3.5 Research Hypotheses

H1. For the same datasets, a neural network architecture with a YOLOX detection head for note bounding box prediction will achieve a higher F1-score for monopohonic note transcription than does the deterministic CREPE Notes algorithm.

H2. With the rest of the architecture held constant, finetuning the last layer of the CREPE backbone, compared to keeping it frozen, will result in a higher Average Overlap Ratio (AOR) for note segmentation.

H3. Under identical architecture and compute settings, training on a pitch-shifted, augmented version of the ITM-Flute-99 dataset will increase model's F-measure on a held-out test set compared to training only on the raw dataset.

4 Experiments & Results

4.1 Baseline Results

For our baselines, we evaluate the ITM-Flute-99 dataset on CREPE Notes (described in 1) and pYIN (described in 2). We choose CREPE Notes as a baseline because our goal is to create a model that can replace CREPE Notes' deterministic heuristic. We also choose pYIN because it was chosen as one of the baselines for CREPE Notes (Bresin & Falkenberg (2023)), and it would be a good starting point if our model could at least outperform this method. The results for the baseline methods are shown in Figure 2.

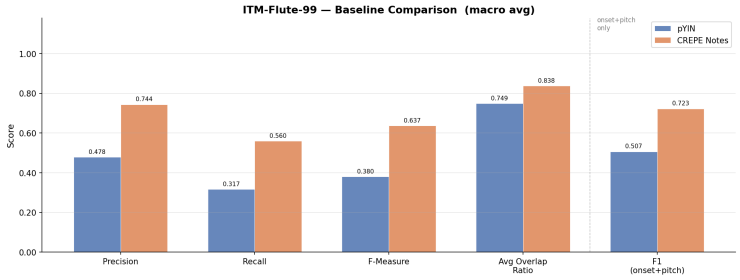

Figure 2: Average Precision, Recall, F-Measure, Overlap Ratio, and F1 Score for CREPE Notes and Pyin.

4.2 Experimentation Results

Evaluating the results from an initial grid-search of hyperparameters, we identified the hyperparameter set which produced the best results over the validation set (learning_rate: 0.0005, weight_decay: 0.0, batch_size: 4, grad_clip: 3.0). For this hyperparameter set, we analyzed the loss and metric curves Figure 3. Here we see that the loss and all metrics appear to plateau in a fairly small number of 20 epochs. Accessing the generalization gap, we see that the model generalized extremely well, and there is no sign of overfitting thus far. This suggests either a lack of model capacity, or a need for continued training. The clear loss plateau seems to indicate the former, but we plan to test this with a longer training run as we move forward.

5 Next Steps

5.1 Action Plan

- Filosax dataset access **(indeterminate timeline)**
  - Two of our group members have placed a request to access Filosax through Zenodo, but have not received a response from the dataset's admin. We have also emailed the dataset's admin directly to their Queen Mary University of London email address, but have not received a response.
  - For the meantime, we augmented the ITM-Flute-99 dataset via pitch modulation and tempo shifts on training samples.
  - The training set size ablation wasn't entirely conclusive due to the peak at 60% training set size, but we still believe there is large room for potential improvement with the filosax dataset.

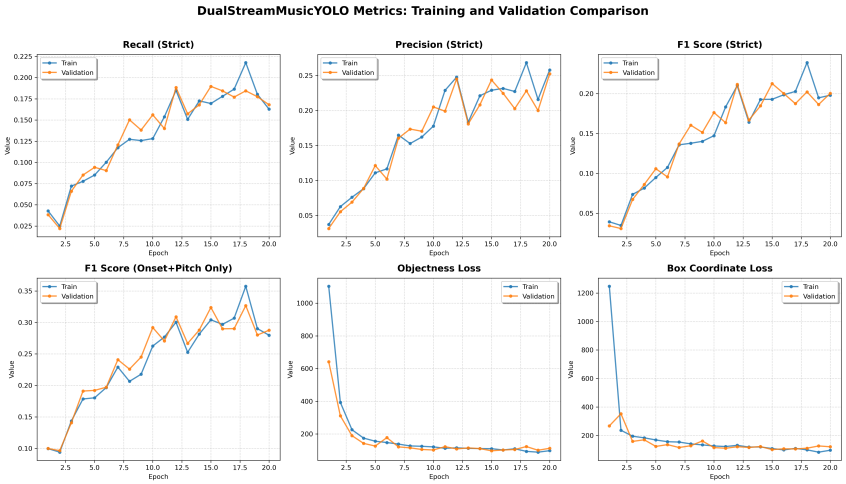

Figure 3: Precision, recall, F1 score, objectness loss, and IoU/box coordinate loss for training and validation of best hyperparameters during tuning.

- Further hyperparameter tuning for CREPE-YOLO training **(goal 4/3 completion)**
  - We would like to attempt a longer training run (more epochs) to see if we can observe overfitting.
  - We would like to implement a learning rate scheduler, as we believe this would allow loss to continue to decrease where we see a plateau now.
- Adjust architecture
  - From YOLO to YOLOX **(goal 4/10 completion)**
    - For the initial implementation, we utilized YOLO which requires anchors, is less generalizable, and has a higher tendency to ignore difficult to identify notes to avoid loss penalties. Implementing YOLOX should help to alleviate these issues.
  - Include raw audio shape after CREPE **(goal 4/17 completion)**
    - CREPE is helpful the f1 audio signal separation, but the output loses timbre, and any other aspects of an instrument's starting and ending note behaviors such as swelling in, fading out, or intonation fluctuation.
    - Creating a skip connection allowing YOLO(X) to also see the raw waveform will provide this context that CREPE output lacks.
  - Increase temporal receptive field **(goal 4/24 completion)**
    - Currently, we are looking at the posteriogram purely as an image, ignoring that the x-axis is time.
    - We believe adding some form of BiLSTM or attention mechanism has potential to take advantage of the time series patterns.
- Fine tune CREPE's last layer(s) **(goal 5/1 completion)**
  - If the the current limitation truly is model capacity, and other previous modifications don't fix this, then unfreezing and fine-tuning the last layer(s) of CREPE should provide plenty of additional model capacity to allow for model lift.

5.2 Success Criteria

The greatest success for us would be to beat the CREPE-Notes baseline results over the same train/val/test split. Another criteria we would consider success is to find sufficient evidence that even with the modifications from the action plan above, this architecture under performs CREPE-Notes.

REFERENCES

Roberto Bresin and Kjetil Falkenberg. Proceedings of the 20th sound and music computing conference, 2023. URL https://zenodo.org/record/8136568.

Dave Foster and Simon Dixon. Filosax: A dataset of annotated jazz saxophone recordings. In *International Society for Music Information Retrieval (ISMIR) Conference*, 2021.

Josh Gardner, Ian Simon, Ethan Manilow, Curtis Hawthorne, and Jesse Engel. Mt3: Multi-task multitrack music transcription, 2022. URL https://arxiv.org/abs/2111.03017.

Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: exceeding YOLO series in 2021. *CoRR*, abs/2107.08430, 2021. URL https://arxiv.org/abs/2107.08430.

Jong Wook Kim, Justin Salamon, Peter Li, and Juan Pablo Bello. Crepe: A convolutional representation for pitch estimation. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 161–165. IEEE Press, 2018. doi: 10.1109/ICASSP.2018.8461329. URL https://doi.org/10.1109/ICASSP.2018.8461329.

M. Kököer, I. Ali-MacLachlan, D. Kearney, et al. Curating and annotating a collection of traditional irish flute recordings to facilitate stylistic analysis. *International Journal on Digital Libraries*, 20 (2):107–121, 2019. doi: 10.1007/s00799-019-00267-2.

Taegyun Kwon, Dasaem Jeong, and Juhan Nam. Polyphonic piano transcription using autoregressive multi-state note model, 2020. URL https://arxiv.org/abs/2010.01104.

Matthias Mauch and Simon Dixon. Pyin: A fundamental frequency estimator using probabilistic threshold distributions. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 659–663, 2014. doi: 10.1109/ICASSP.2014.6853678.

Xianke Wang, Wei Xu, Weiming Yang, and Wenqing Cheng. Musicyolo: A sight-singing onset/offset detection framework based on object detection instead of spectrum frames. In *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 396–400, 2022. doi: 10.1109/ICASSP43922.2022.9746684.